

the message echoed back by the server. Let's have a look at a simple attack scenario to understand it clearly.

Attack scenario

Let's continue with the example that we used (for reflected XSS) in Part 2 of this series. This had an attacker finding an XSS vulnerability in a Web application, where the application's script displayed the criteria used in a website query as part of the URL for the results of the query. For example, the URL in the browser for a page showing the results for a search for 'products', might be <http://www.example.com/search.php?query=products>. Now, let's look at the following cases:

1. When `HttpOnly` cookies are not deployed, and `TRACE` is enabled: Here, as already discussed in Part 2, the attacker might post a modified link, such as this:

```
http://www.example.com/search.php?query=<script>alert(document.cookie)</script>
```

This harmless example will cause the victim's browser to pop up an alert box, showing the current set of cookies.

2. When `HttpOnly` cookies are deployed but `TRACE` is still enabled: Here, it is clear that the above method will not work for the attacker, because `HttpOnly` will not return these cookies to the JavaScript `document.cookie`. So here, the attacker, knowing that `TRACE` is enabled on the Web server (this can be verified by methods given in the security section below), might use something like the following code (call it `mal.js`):

```
<script type="text/javascript">
var x = new ActiveXObject("Microsoft.XMLHTTP");
// var x = new XMLHttpRequest();
x.open("TRACE", "http://example.com", false);
x.send();
//x.send("");
cookie=x.responseText;
alert(cookie);
</script>
```

(The code above is used for Internet Explorer browsers; modifications required for Mozilla are commented out.) This will alert the victim with his current set of cookies even after `HttpOnly` is used. Remember, here the attacker can shorten such a big malicious code via tiny URL techniques (refer to Part 2 of this series). The attacker can also steal the cookies and other login credentials. A visualisation of this is shown in Figure 1.

The particular example shown above has been used just for educational purposes. I once again stress that neither LFI nor I aim to teach readers how to attack servers. Rather, the attack techniques are meant to give you the knowledge that you need to protect your own infrastructure.

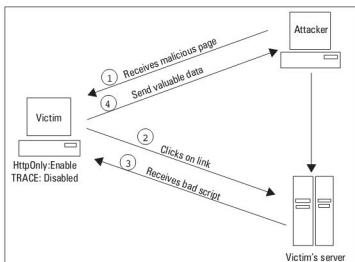


Figure 1: Attack possible with TRACE enabled

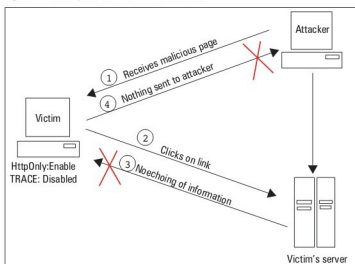


Figure 2: Attack not possible with TRACE disabled

What's going on behind the scenes?

The code above that uses the ActiveX control `XMLHTTP`, will send a `TRACE` request to the target Web server. If `TRACE` is enabled on the Web server, it will then echo the information sent within the HTTP request. Now, if the victim's browser happens to have a cookie from the target Web server, or is logged in to the server using implicit authentication mechanisms, then the victim will be alerted with his current set of cookies.

What if you disable TRACE?

Many security experts suggest that you disable the Web server's `TRACE` method to begin with, though it provides an effective security measure (see Figure 2). However, if there is a proxy server between the client and the Web server, it has been found possible to force the proxy server to respond to the `TRACE` request, rather than the origin server itself. To do this, the attacker would simply include `'Max-Forward: 0'` in the HTTP request header. Seeing this, the first proxy server in the chain will respond to the `TRACE` request, instead of forwarding it to the Web server. Hence, the XSS script could be updated to: